> **Fuente original:** https://algoritmos-rw.github.io/tda_bg/material/guias/pd/

Programación Dinámica

Contenidos

1. Ejercicio resuelto
2. Ejercicios propuestos

Ejercicio resuelto

Solución

Demostración del orden

Ejercicios propuestos

1. (★) Implementar un algoritmo que, utilizando programación dinámica, obtenga el valor del n-ésimo número de fibonacci. Indicar y justificar la complejidad del algoritmo implementado. Definición:

```
n = 0 --> Debe devolver 1
n = 1 --> Debe devolver 1
n --> Debe devolver la suma entre los dos anteriores números de fibonacci (los fibonacci n-2 y n-1)
```

2. (★★★) Dada un aula/sala donde se pueden dar charlas. Las charlas tienen horario de inicio y fin. Además, cada charla tiene asociado un valor de ganancia. Implementar un algoritmo que, utilizando programación dinámica, reciba un arreglo que en cada posición tenga una charla representada por una tripla de *inicio, fin y valor* de cada charla, e indique cuáles son las charlas a dar para maximizar la ganancia total obtenida. Indicar y justificar la complejidad del algoritmo implementado.

3. (★) Dada una escalera, y sabiendo que tenemos la capacidad de subir escalones de a 1 o 2 o 3 pasos, encontrar, utilizando programación dinámica, cuántas formas diferentes hay de subir la escalera hasta el paso n. Indicar y justificar la complejidad del algoritmo implementado. Ejemplos: n = 0 –> Debe devolver 1 (no moverse) n = 1 –> Debe devolver 1 (paso de 1) n = 2 –> Debe devolver 2 (dos pasos de 1, o un paso de 2) n = 3 –> Debe devolver 4 (un paso de 3, o tres pasos de 1, o un paso de 2 y uno de 1, o un paso de 1 y un paso de 2) n = 4 –> Debe devolver 7 n = 5 –> Debe devolver 13

4. (★★) Juan es ambicioso pero también algo vago. Dispone de varias ofertas de trabajo diarias, pero no quiere trabajar dos días seguidos. Dado un arreglo con el monto esperado a ganar cada día, determinar, por programación dinámica, el máximo monto a ganar, sabiendo que no aceptará trabajar dos días seguidos. Hacer una reconstrucción para verificar qué días debe trabajar. Indicar y justificar la complejidad del algoritmo implementado.

5. (★) Dado un laberinto representado por una grilla, queremos calcular la ganancia máxima que existe desde la posición `(0,0)` hasta la posición NxM. Los movimientos permitidos son, desde la esquina superior izquierda (el `(0,0)`), nos podemos mover hacia abajo o hacia la derecha. Pasar por un casillero determinado (i, j) nos da una ganancia de $V_{i,j}$. Implementar un algoritmo que, por programación dinámica, obtenga la máxima ganancia a través del laberinto. Hacer una reconstrucción del camino que se debe transitar. Indicar y justificar la complejidad del algoritmo implementado. Si hay algunos lugares por los que no podemos pasar (obstáculos), ¿cómo se debe modificar para resolver el mismo problema?

6. (★★) Dado el teclado numérico de un celular, y un número inicial k, encontrar la cantidad de posibles números de longitud n empezando por el botón del número inicial k. Restricción: solamente se puede presionar un botón si está arriba, abajo, a izquierda, o derecha del botón actual. Implementar el algoritmo por programación dinámica. Indicar y justificar la complejidad del algoritmo implementado. Ejemplos:

   Para n=1 empezando por cualquier dígito, solamente hay un número válido (el correspondiente dígito)

   Para N=2, depende de con cuál dígito se comienza:

   Empezando por 0, son válidos 08 (cantidad: 1)

   Empezando por 1, son válidos 12, 14 (cantidad: 2)

   Empezando por 2, son válidos 21, 23, 25 (cantidad: 3)

   Empezando por 3, son válidos 32, 36 (cantidad: 2)

   Empezando por 4, son válidos 41, 45, 47 (cantidad: 3)

   Empezando por 5, son válidos 52, 54, 56, 58 (cantidad: 4)

Empezando por 6, son válidos 63, 65, 69 (cantidad: 3)

Empezando por 7, son válidos 74, 78 (cantidad: 2)

Empezando por 8, son válidos 80, 85, 87, 89 (cantidad: 4)

Empezando por 9, son válidos 96, 98 (cantidad: 2)

7. (★★★) Tenemos una mochila con una capacidad W. Hay elementos a guardar, cada uno tiene un valor, y un peso que ocupa de la capacidad total. Queremos maximizar el valor de lo que llevamos sin exceder la capacidad. Implementar un algoritmo que, por programación dinámica, reciba los valores y pesos de los elementos, y devuelva qué elementos deben ser guardados para maximizar la ganancia total. Indicar y justificar la complejidad del algoritmo implementado.

8. (★★) Se tiene un sistema monetario (ejemplo, el nuestro). Se quiere dar “cambio” de una determinada cantidad de plata. Se desea devolver el cambio pedido, usando la mínima cantidad de monedas/billetes. Implementar un algoritmo que, por programación dinámica, reciba un arreglo de valores del sistema monetario, y la cantidad de cambio objetivo a dar, y devuelva qué monedas/billetes deben ser utilizados para minimizar la cantidad total utilizda. Indicar y justificar la complejidad del algoritmo implementado.

9. (★★★) Tenemos un conjunto de números $v_1, v_2, \cdots, v_n$, y queremos obtener un subconjunto de todos esos números tal que su suma sea igual o menor a un valor V, tratando de aproximarse lo más posible a V. Implementar un algoritmo que, por programación dinámica, reciba un arreglo de valores, y la suma objetivo V, y devuelva qué elementos deben ser utilizados para aproximar la suma lo más posible a V, sin pasarse. Indicar y justificar la complejidad del algoritmo implementado.

10. (★★★) Manejamos un negocio que atiende clientes en Londres y en California. Nos interesa cada mes decidir si operar en una u otra ciudad. Los costos de operación para cada mes pueden variar y son dados por 2 arreglos: L y C, con valores para todos los meses hasta n. Naturalmente, si en un mes operamos en una ciudad, y al siguiente en una distinta, habrá un costo fijo M por la mudanza. Dados los arreglos de costos de operación en Londres (L) y California (C), indicar la secuencia de las n localizaciones en las que operar durante los n meses, sabiendo que queremos minimizar el total de los costos de operación. Se puede empezar en cualquier ciudad. Indicar y justificar la complejidad del algoritmo implementado.

11. (★★) Dado un número **K**, se quiere obtener la mínima cantidad de operaciones para llegar desde 0 a K, siendo que las operaciones posibles son:

    (i) aumentar el valor del operando en 1;

    (ii) duplicar el valor del operando.

    Implementar un algoritmo que, por programación dinámica, obtenga la menor cantidad de operaciones a realizar (y cuáles son dichas operaciones). Desarrollar la ecuación de recurrencia. Indicar y justificar la complejidad del algoritmo implementado. Aclaración: asegurarse de que el algoritmo presentado sea de programación dinámica, con su correspondiente ecuación de recurrencia.

12. (★★★) Carlitos (primo de Juan, el vago) trabaja para una empresa de publicidad. Tiene un determinado presupuesto P que no puede sobrepasar, y tiene que una serie de campañas publicitarias para elegir. La campaña i cuesta $\$C_i$. También se han realizado diversos estudios que permiten estimar cuánta ganancia nos dará cada campaña, que denominaremos G_i. Implementar un algoritmo que reciba esta información y devuelva cuáles campañas debe realizar Carlitos. Indicar y justificar la complejidad del algoritmo propuesto. ¿Da lo mismo si los valores están expresados en pesos argentinos, dólares u otra moneda? Por ejemplo, si una campaña cuesta 100 dólares, para pasar a pesos se debe hacer la conversión de divisa.

13. (★★★) Un bodegón tiene una única mesa larga con **W** lugares. Hay una persona en la puerta que anota los grupos que quieren sentarse a comer, y la *cantidad de integrantes* que conforma a cada uno. Para simplificar su trabajo, se los anota en un vector **P** donde P[i] contiene la cantidad de personas que integran el *grupo i*, siendo en total **n** grupos. Como se trata de un restaurante familiar, las personas sólo se sientan en la mesa si *todos los integrantes de su grupo* pueden sentarse. Implementar un algoritmo que, mediante programación dinámica, obtenga el conjunto de grupos que ocupan la mayor cantidad de espacios en la mesa (o en otras palabras, que dejan la menor cantidad de espacios vacíos). Indicar y justificar la complejidad del algoritmo.

14. (★★★) Somos ayudantes del gran ladrón *el Lunático*, que está pensando en su próximo atraco. Decidió en este caso robar toda una calle en un barrio privado, que tiene la particularidad de ser circular. Gracias a los trabajos de inteligencia realizados, sabemos cuánto se puede obtener por robar en cada casa. Podemos enumerar a la primer casa como la casa 0, de la cual podríamos obtener g_0, la casa a su derecha es la 1, que nos daría g_1, y así hasta llegar a la casa $n-1$, que nos daría g_{n-1}. Toda casa i se considera adyacente a las casas $i-1$ e $i+1$. Además, como la calle es circular, la casas 0 y $n-1$ también **son vecinas**. El problema con el que cuenta *el Lunático* es que sabe de experiencias anteriores que, si roba en una casa, los vecinos directos se enterarían muy rápido. No le daría tiempo a luego intentar robarles a ellos. Es decir, para robar una casa debe prescindir de robarle a sus vecinos directos. El Lunático nos encarga saber cuáles casas debería atracar y cuál sería la ganancia máxima obtenible. Dado que nosotros nos llevamos un porcentaje de dicha ganancia, vamos a buscar el óptimo a este problema.

Implementar un algoritmo que, por **programación dinámica**, obtenga la ganancia óptima, así como cuáles casas habría que robar, a partir de recibir un arreglo de las ganancias obtenibles. Para esto, escribir y describir la ecuación de recurrencia correspondiente. Indicar y justificar la complejidad del algoritmo propuesto.

15. (★★★) Dada una soga de n metros ($n \geq 2$) implementar un algoritmo que, utilizando programación dinámica, permita cortarla (en partes de largo entero) de manera tal que el producto de los largos de cada una de las partes resultantes sea máximo. El algoritmo debe devolver el valor del producto máximo alcanzable. Tener en cuenta que la soga puede cortarse varias veces, como se muestra en el ejemplo con n = 10. Indicar y justificar la complejidad del algoritmo. Ejemplos:

    n = 2 –> Debe devolver 1 (producto máximo es 1 * 1)

    n = 3 –> Debe devolver 2 (producto máximo es 2 * 1)

    n = 4 –> Debe devolver 4 (producto máximo es 2 * 2)

    n = 5 –> Debe devolver 6 (producto máximo es 2 * 3)

    n = 6 –> Debe devolver 9 (producto máximo es 3 * 3)

    n = 10 -> Debe devolver 36 (producto máximo es 3 * 3 * 4)

16. (★★★★) Osvaldo es un empleado de una inescrupulosa empresa inmobiliaria, y está buscando un ascenso. Está viendo cómo se predice que evolucionará el precio de un inmueble (el cual no poseen, pero pueden comprar). Tiene la información de estas predicciones en el arreglo p, para todo día $i = 1, 2, ..., n$. Osvaldo quiere determinar un día j en el cuál comprar la casa, y un día k en el cual venderla ($k > j$), suponiendo que eso sucederá sin lugar a dudas. El objetivo, por supuesto, es la de maximizar la ganancia dada por $p[k] - p[j]$.

    Implementar un algoritmo de **programación dinámica** que permita resolver el problema de Osvaldo. Indicar y justificar la complejidad del algoritmo implementado.

17. (★) Se sabe que existe la equivalencia $e = \sum_{i=0}^{\infty} \frac{1}{n!}$, siendo e el número de Euler. Sabiendo esto, podemos aproximar el valor de e sumando los primeros n términos de dicha serie. Un algoritmo trivial puede obtener esto en $\mathcal{O}(n^2)$. Implementar un algoritmo que, utilizando programación dinámica, permita obtener dicha aproximación en tiempo lineal.